Development of a multimodal AI for windows-APT attack recognition

Student n°8510 and Student n°8503

University of South-Eastern Norway, Kongsberg 3616, Norway

Abstract. Traditional signature-based intrusion detection systems fail to identify novel attack techniques, and even Security Information and Event Management (SIEM) platforms, while effective at aggregating and correlating security events, still rely heavily on manual analyst effort. This paper presents a two-stage deep learning pipeline that automates both the detection of cyberattacks and the identification of the corresponding MITRE ATT&CK tactic from Wazuh SIEM logs. Raw security events are transformed into fixed-length overlapping sequences of 15 events using a sliding window approach, capturing the temporal patterns that characterise multi-step attack behaviour across 228 engineered features. The first stage, a two-layer Long Short-Term Memory (LSTM) network, performs binary classification of event sequences as benign or malicious, achieving approximately 97% accuracy and an F1-score of 0.97 for both classes. Sequences flagged as attacks are forwarded to the second stage, a three-layer 1D Convolutional Neural Network with an attention mechanism, which classifies the sequence into one of the MITRE ATT&CK tactics with 81% of accuracy. The full pipeline is evaluated on a strictly chronological train/validation/test split, simulating real deployment conditions where the model predicts on future, previously unseen data. End-to-end evaluation on 2,972 test sequences demonstrates that the pipeline provides both reliable attack detection and actionable tactic-level intelligence, offering a practical and scalable contribution to automated threat detection in operational SOC environments.

Keywords: MITRE ATT&CK · Intrusion Detection · Tactics Techniques and Procedures · Cybersecurity Dataset · Classification.

Use of AI

Throughout this project, large language models (Perplexity AI and Claude Sonnet 4.6) were used as development aids for three specific purposes: brainstorming architecture variants and justifying design choices and generating starter code for EDA, preprocessing and helping to interpret error messages and debug implementation issues. In all cases, generated code and suggestions were critically reviewed, tested, and adapted before use. No results, metrics, or conclusions in this report were produced by or delegated to an AI tool. Also all experiments were run independently and all reported values come from our actual model training and evaluation.

1 Introduction

Monitoring is a predominant part in the Cybersecurity of IT and OT, but the threat landscape that organisations are facing increased significantly over the past decade. Cybercriminals increasingly rely on more Advanced Persistent Threats (APTs), multi-stage, stealthy campaigns that blend into normal network activity and can persist for weeks or months before being discovered. The Security Operations Center (SOC) team cannot use traditional signature-based intrusion detection systems (IDS) because they are not adapted to counter these threats. They can only flag known attack patterns and may not detect novel types of attack while generating enormous volumes of alerts that represent a huge need of triage for security analysts.

Security Information and Event Management (SIEM) platforms, such as Wazuh, attempt to address this by aggregating, normalising, and correlating events from across the infrastructure into a single stream of security logs. However, even with an SIEM in place, distinguishing malicious activity from benign noise remains a largely manual and reactive process, often without clear guidance on which attack stage is underway or how urgently to respond.

The MITRE ATT&CK framework provides a structured vocabulary of adversarial tactics (high-level objectives such as Execution, Lateral Movement, or Exfiltration) and techniques (the specific methods used to achieve them). Mapping a detected attack to an ATT&CK tactic gives defenders all the information to understand the intentions of the cybercriminal and helps prioritise response. However, automatically predicting these tactic labels from raw SIEM logs in real time and across many different attack types, remains an open and challenging problem.

This project contribute to help SOC with triage and response by addressing these limitations directly. Given a continuous stream of Wazuh SIEM events, using two deep learning models that can be summarise with two questions: *Is this sequence of events part of an attack?* If so, *which MITRE ATT&CK tactic does it correspond to ?* In order to achieve this, we designed a deep learning pipeline that models Wazuh logs (TF-IDF on command lines, categorical encodings, sliding windows) as time-ordered sequences, capturing the temporal patterns that characterise multi-step attack behaviour, and applies a two-stage architecture: an LSTM network for binary detection followed by a 1D-CNN with an attention mechanism for tactic classification.

2 Existing literature

As the complexity of cyber threats and attacks is increasing, the detection and identification of anomalies in network traffic become increasingly complex. In this context, Intrusion Detection Systems (IDS) are helpful weapons to help us detect these intrusions. To be able to train IDS based on Machine Learning models to detect increasingly diverse attacks, huge network traffic and log datasets are needed.

One of the first complete intrusion detection dataset is the *1998 DARPA Intrusion Detection Evaluation Dataset* [1]. This dataset has been a reference for many years because it presents among the widest range of network traffic attack cases when it was realised. Even if it can still be useful, this dataset has been considered outdated for the last decade. Indeed, this dataset does not reflect today's traffic complexity and low footprints attacks.

Resulting to a updated new dataset in 2015 made by the University of New South Wales with 100Gb Network Intrusion Detection data : *UNSW-NB15*[2]. It compiles both modern and contemporary attack activities on network traffic. With the same goal, *CIC-IDS2017*[3] has been published in 2017 with a more realistic 50Gb network traffic dataset. However these datasets are only focused on network intrusion. Newer dataset such as Linux-APT[4] imitate APT behavior, but only on Linux systems. As windows systems are more widespread, APTs against Windows environment are more likely to append, explaining why we decided to work with the Windows-APT 2025 [5] dataset instead.

Recent machine learning models made for IDS are mostly made with the same base : a combination of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM). A hybrid model combining CNN and Weight-Dropped LSTM [6], trained on UNSW-NB15, has shown 97.17% accuracy on detecting whether a pattern is an attack or not. For the attack classification they registered $98,34\%$ accuracy. This result is extremely high, but normal patterns have been kept in this classification, so the results are biased by the vast majority of normal patterns present among the samples. In 2025 a new model using Attention-CNN and LSTM, trained on NSL-KDD and Bot-IoT datasets, has shown impressive results[7]. These results suggest that adding an Attention layer to CNN-LSTM contributes to overall performance gains.

Table 1: Summary of existing Datasets

Reference	**Content**	**Limitations**
1998 DARPA Intrusion Detection Evaluation Dataset [1]	Serveral weeks of network traffic with normal and malicious activities	Focused on network traffic only, limited at host level.
UNSW-NB15: a comprehensive data set for network intrusion detection systems [2]	31h of network traffic, compiling 9 attack families	Ultra heavy dataset (100Gb), focused on network traffic only, limited at host level.
Intrusion detection evaluation dataset (CIC-IDS2017) [3]	5 days complete network traffic with normal activity and 21 different attack scenarios	Heavy dataset (51,1Gb), focused on network intrusion attacks
APT and intrusion detection evaluation dataset for linux 2024 [4]	APT attacks on Linux	Focuses on Linux so it is limited to techniques used on Linux system.

Table 2: Summary of existing works

Reference	**Methodology**	**Limitations**
A hybrid deep learning model for efficient intrusion detection in big data environment (2020) [6]	CNN-WDLSTM : Hybrid model combining Deep CNN with Weight-Dropped LSTM. Dataset Used : UNSW-NB15	Kept benigns in attack classification, precision results are biaised.
Deep learning for network security: an Attention-CNN-LSTM model for accurate intrusion detection (2025) [7]	Attention-CNN-LSTM : Hybrid model combining CNN and LSTM networks with added Attention Layer on top of it. Datasets used : NSL-KDD, Bot-IoT	Test made on outdated traffic datasets. Model's efficacy in real-world traffic must be studied
CNN-LSTM: Hybrid Deep Neural Network for Network Intrusion Detection System (2025) [8]	CNN-LSTM : Hybrid model combining CNN and LSTM. Dataset used : CIC-IDS 2017, UNSW-NB15, and WSN-DS	Uses 3 heavy datasets, huge data preprocessing, difficult to replicate.

3 Materials and Methods

3.1 Dataset

The dataset used in this project is a publicly available collection of APT simulation logs mapped to the MITRE ATT&CK framework, sourced from the Mendeley Data repository [5]. It contains scenarios inspired by 36 APT that come from MITRE ATT&CK threat actor profiles, all recreated in a controlled Windows 10 environment instrumented with the Wazuh SIEM agent, capturing the full event stream with both malicious activity and background benign traffic throughout all simulation periods.

The dataset is mainly organised into three files, each serving a distinct role in the training, testing and validation. The first one is *combined.csv*, the primary log file, merging all simulation periods into a single chronologically ordered event stream. Each row represents one security event as captured by Wazuh. This is the main input of the models. The second one is *scenario_manifest.csv*, this datafile enables ATT&CK-driven and scenario-level analysis by filtering scenarios with MITRE ATT&CK tactics or techniques, constructing targeted evaluation subsets for intrusion detection research. The last one is *validation_summary.csv*, it is summarising execution consistency across repeated runs, highlighting scenarios with stable and well-observed behaviour, and comparing expected versus observed technique execution rates.

3.2 Pre-processing

Label encoding : Two label types are derived from the dataset and used at different stages of the pipeline. The first one is a binary label that indicates whether an event belongs to a malicious activity or to benign background traffic. It is derived directly from the MITRE tactic column, Attack (1) if the source is non-empty and non-null and Benign (0) otherwise. The second label is the multi-class label, identifying which of the 25 MITRE ATT&CK tactics an attack event belongs to. It is derived from the *source.rule.mitre.tactic* column and encoded into consecutive integers using *LabelEncoder* from *scikit-learn*. Examples of tactic classes present in the dataset include:

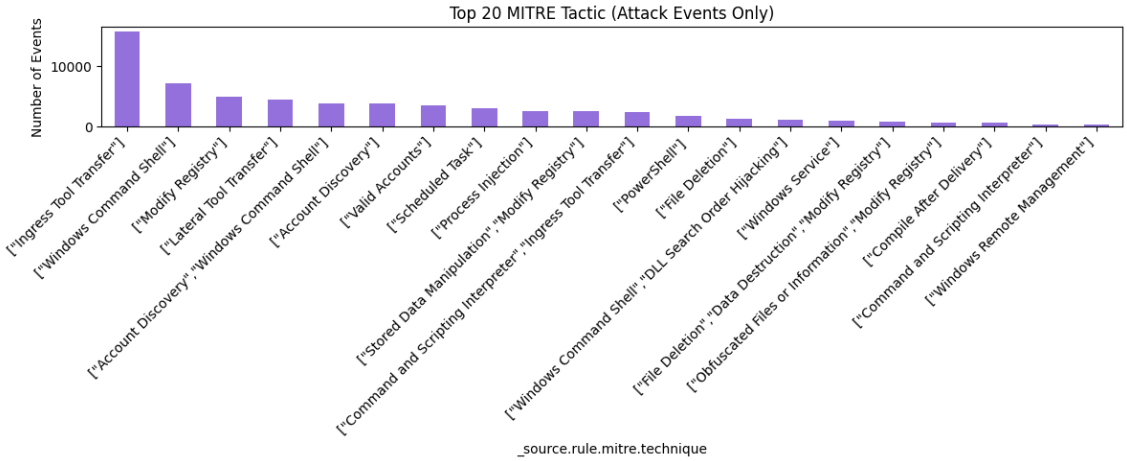

Figure 1 : repartition of tactic class in dataset

It is important to note that tactic labels in the dataset are not mutually exclusive. A single event can be tagged with multiple tactics. In such cases, the primary tactic is used as the target label.

Column section and dropping : Two categories of columns are removed. First, the Elasticsearch metadata columns, which contains system identifiers with no predictive content such as index, id, version, and score. Next, the High-sparsity columns, meaning any column with more than 80% missing values, as imputation on such columns introduces more noise than signal, and oversampling them would introduce more noise than anything else. This step removes a significant portion of the 300+ raw columns, leaving a manageable set of informative fields.

Missing value imputation : For the remaining columns, the numerical columns were filled with the median of the training set and the text columns we replace with "unknown", avoiding confusion between missing value and any real category.

Categorical encoding : Categorical columns fall into two groups based on their cardinality. High-cardinality columns are encoded with LabelEncoder, mapping each unique string to an integer, and low-cardinality columns encoded with one-hot encoding, producing a sparse binary matrix that avoids imposing an artificial ordinal relationship between categories.

Outlier handling : For the outlier detection, we used an Isolation Forest with a contamination parameter of 5% applied to the benign event subset of the training data. Detected outliers are removed from training even if attack events are never removed by this step, as even unusual attack samples carry valuable signal.

TF-IDF feature extraction from command-line text : Two text fields contain rich attack content : the *source.data.win.eventdata.commandLine* and the *source.data.win.eventdata.parentCommandLine*. Instead of discarding these, a Term Frequency-Inverse Document Frequency (TF-IDF) vectoriser is fitted on top 100 features per field, extracting keyword-based numerical features. This step captures tokens that are strongly associated with specific attack behaviours as *powershell, certutil, reg, mimikatz, net use, whoami*, and converts raw command text into 200 additional numeric dimensions usable by the neural network.

Feature scaling : All numeric columns are scaled to the range using scikit-learn's MinMaxScaler. This ensures that no single feature dominates gradient updates during training due to differences in raw scale. The scaler is fitted only on the training set.

3.3 Sequence Construction with Sliding Windows

Raw security logs are inherently sequential and heavily rely on time, an attack campaign unfolds across multiple events over time, and no single event in isolation is sufficient to reliably identify a tactic. For example, a process creation event may not be suspicious on its own, but when it comes after a suspicious logon event and followed by a network connection to an unusual destination, it becomes a strong indicator of lateral movement. To capture this temporal context, each event stream is transformed into overlapping fixed-length sequences using a sliding window approach.

After sorting all events chronologically by *source.timestamp*, a window of size 15 is slid over the event stream with a step of 5, meaning consecutive windows overlap by 10 events. Based on the paper of S. Meganathan et al. [9], formally for a sorted event sequence $e_1, e_2, ..., e_N$, window w_k is defined as:

$$w_k = \sum_{i}^{\text{window size}} e_i + (k-1) \times step \tag{1}$$

$$w_k = (e_1 + (k-1) \times 5,\ e_2 + (k-1) * 5, ..., e_{15} + (k-1) \times 5) \tag{2}$$

Each window produces a matrix of shape (15,228), 15 time steps, each with 228 features, which serves as a single input sample to either the LSTM or CNN model. Applying this procedure to the entire event stream gives us 19,800 sequences in total. For the parameters, a wider window captures longer-range temporal dependencies but reduces the total number of samples and may blend events from different attack phases. A smaller window is more fine-grained but may lack sufficient context for reliable classification. We choose a size of 15 with a step of 5 as a practical balance.

Each window has a label, if at least one event within it carries an attack label then the windows will be labelled Attack (1), otherwise it is labelled Benign (0). This is a conservative labelling that ensures no attack event goes unrepresented. Regarding the technique, when a window is labelled as attacks (1), the tactic is assigned as the majority tactic among the attack events within the window. In cases of equality, the first occurring tactic is used.

It is important to emphasise that windowing is performed separately on each split after the chronological train/val/test division, preventing sequences from spanning the boundary between splits and introducing future information into training windows.

3.4 Train / Validation / Test Split

A fundamental design constraint of this project is that the model must generalise to future, unseen data, as would be required in any real operational deployment. This rules out random shuffling of samples before splitting but creates a form of temporal data leakage in time-series settings. If future events appear in the training set, the model benefits from information it could not have in production. Instead, the dataset is split chronologically in the following proportions:

Table 3: Data split repartition

Split	**Proportion**	**Role**
training set	70%	Model fitting, augmentation, and class weight
Validation set	15%	Hyperparameter tuning, early stopping
Test set	15%	Final, unbiased performance evaluation

The split is applied to the ordered sequence of windows, so the training set contains the first 70% of windows, the validation set the next 15%, and the test set the final 15%. The goal was to imitate real case deployment where the model is trained on historical data and evaluated on events it has never encountered, drawn from a later time period.

3.5 Architecture

Our system is designed as a two-stage sequential pipeline. The idea was to first determine whether something suspicious is happening and then characterise what the attacker is doing. Binary detection is a simple problem with a balanced signal once the windows have been constructed. Tactic classification is a lot harder; the best to do is to train a fine-grained multi-class classification model on the subset of sequences already flagged as attacks. This can avoid having biased results where CNN has high accuracy because it correctly labels benign events, which represent an overwhelming majority of events in real world, while having poor accuracy in attack detection. It also allows us to train, tune, and evaluate each model independently. Having both LSTM and CNN allows the LSTM model to filter a lot of data for the CNN, thereby reducing data requirements.

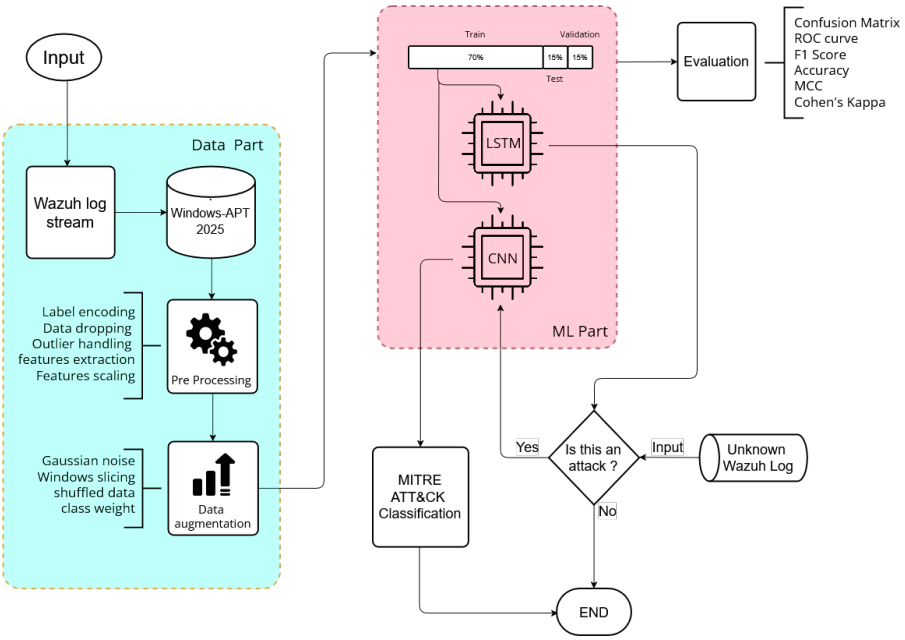

Figure 2 : Proposed architecture flow chart

Model 1, Long Short-Term Memory : The first model is a two-layer Long Short-Term Memory (LSTM) network, because of its ability to model sequential dependencies in time-series data. LSTMs are particularly suited to this task because they maintain a hidden state across time steps, allowing them to capture patterns that span multiple events within a window, for example, the progression from a reconnaissance event to a privilege escalation attempt.

Table 4: Model 1 parameters

Parameter	**Value**
Loss function	binary cross-entropy.
validation function	relu, Sigmoid
Epoch	30
Optimiser	Adam
learning rate	0.001
Early stopping	monitored on validation AUC with a patience of 5 epochs

Model 2, Convolutional Neural Network with attention : The second model is a 1D CNN with an attention mechanism, trained exclusively on sequences already identified as attacks. CNNs are efficient in extracting local feature patterns across the time dimension, such as a characteristic combination of event types that distinguishes Lateral Movement from Credential Access, while the attention mechanism allows the model to assign higher importance to the most discriminative time steps within a window, rather than treating all positions equally.

Table 5: Model 2 parameters

Parameter	**Value**
Loss function	sparse categorical cross-entropy.
validation function	ReLu, tanh
Epoch	100
Optimiser	RMSprop
learning rate	0.001
Early stopping	monitored on validation acc. with a patience of 7 epochs

For each model, a Hyperparameter Tuner has been run, testing different combinations of layer size, dropout value, and learning rate using *GridSearch* , allowing us to determine the best global parameters for our models.

3.6 Data Augmentation

Logs are inherently imbalanced at two levels, the benign events represent the majority of logs because attacks are still rare, and among attack events, some tactics appear in only a handful of windows after the chronological split. So we

do a three-step augmentation and balancing strategy applied exclusively to the training set.

Jitter : Some Gaussian noise with a small standard deviation is independently added to every numeric feature of every training sequence. This doubles the size of the training set by creating a perturbed copy of each original sample, helping to reduce overfitting to exact feature values while preserving the overall structure of each sequence.

Window slicing : Each training sequence is further augmented by randomly cropping a contiguous sub-sequence and rescaling it back to the original window length of 15 time steps using linear interpolation. Two augmented copies are generated per original sample, which triples the post-jitter dataset size. Window slicing encourages the model to recognise attack patterns that may start or end at different positions within a window, improving robustness to temporal shifts.

Random Under Sampling : For the CNN, we applied random Under Sampling to dominant classes, capping each class at the median class size. This prevents the largest classes from overwhelming the loss function and forcing the model into a mode of predicting only common tactics.

3.7 Model evaluation

To evaluate these models, we use multiple common evaluation metrics, such as :

- Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$
- Precision $= \frac{TP}{TP+FP}$
- Recall $= \frac{TP}{TP+FN}$
- $F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$
- Matthew Correlation Coefficient (MCC) $= \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$
- Cohen's Kappa (k) $= \frac{P_o - P_e}{1 - P_e}$

These metrics enabled a comprehensive evaluation or each model strategy and architecture.

4 Results and discussion

4.1 Results

LSTM Model : The 2 models presented previously were assessed on the Windows-APT 2025 dataset. For the binary classification, the initial goal was to have a minimum of undetected attacks by focusing on recall. To address this

problem, the model started to label everything as an attack, making it impossible to keep the focus on recall. The decision was made to focus on precision instead, limiting the number of undetected attacks while maintaining a good overall score.

Table 6: LSTM Model results

	Precision	**Recall**	**f1-Score**	**Support**
Benign	0.9821	0.9684	0.9752	1359
Attack	0.9737	0.9851	0.9794	1613
Accuracy			0.9775	2972

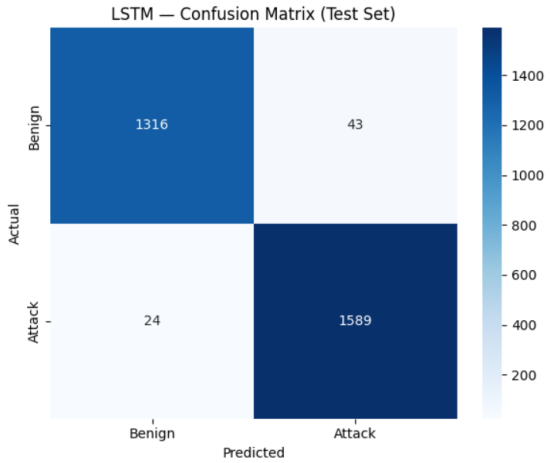

Figure 3 : LSTM confusion matrix

We can see from the above table and confusion matrix that our LSTM model is performing very well, with the best parameters found with the hyperparameter tuner function. This first LSTM model ended up with a strong $97,75\%$ accuracy for the binary classification, with still strong recall. Very few attacks are classified as benign, which is promising and consistent,assuring a possible use in a real case environment.

CNNs model : The multi-class classification went through more experiments to find the best model. The first model was a simple 1D-CNN, as CNN is a great choice for multi-class classification, we wanted to see what results a simple CNN could get. This first model ended-up with 68.20% accuracy. For the second model, as done in several multi-class classification CNN models [7][10], an attention layer was added on top of the CNN. Adding this attention layer has brought our model results to a satisfying accuracy of 71.73%. Finally, for the last model experiment,

an LSTM layer was added before the attention layer, rising the model accuracy up to $81,30\%$.

Regarding the data part of the CNN model, we went through several questions. We wanted to try if it was better to use RandomUnderSampler on the median and remove rare classes (classes with less than 20 events) or if it was better to oversample events using SMOTE, even on the rare classes. At first, we tried not to drop values because that would induce a loss of data. Thereby we decided to oversample rare classes up to 50 events, but ended up with a loss of accuracy of $5-10\%$. This result can be explained by a lack of diversity on those extra rare classes. Indeed, oversampling classes with only 6 events up to 50 can not result in a good diversity among samples, so the model could not recognise rare attacks if the pattern is not the exact same as the oversampled ones. In addition, the ram limitation of our environment prevented us from using larger data for training. That is why we decided, in the end, to simply drop the rare classes.

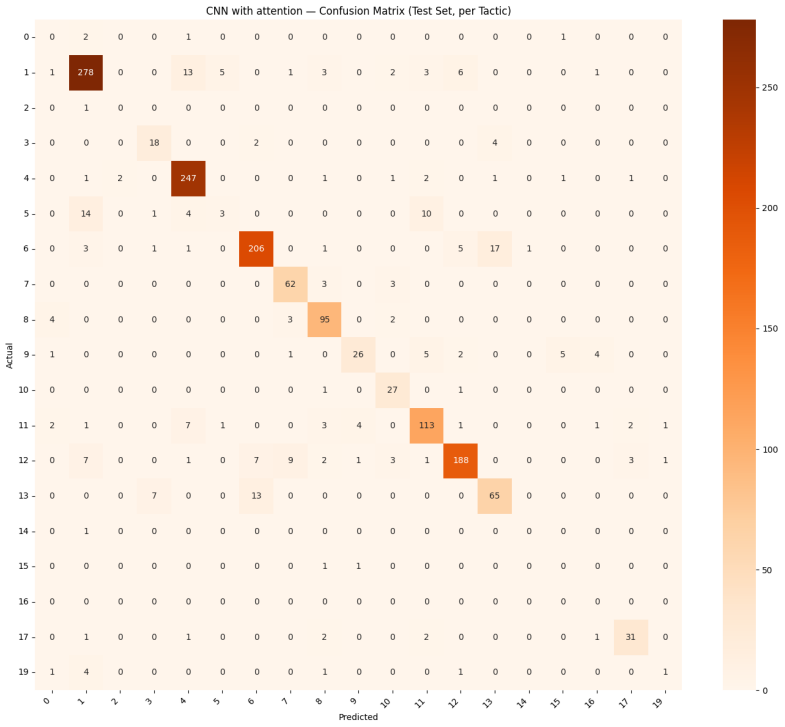

Figure 4 : CNN confusion matrix

Our best CNN model performed $81,30\%$ of accuracy on average for multi-class classification. Thanks to MCC and Cohen's Kappa metrics we can also admit that the result is not random, and our model made real choices and decisions. In addition these performances are really good knowing the number of difference classes with some of them containing very few events.

Table 7: Evaluation metrics for CNNs model

	1D-CNN	**CNN-Attn**	**CNN-Attn-LSTM**
Accuracy	68.20%	73.73%	81.30%
F1-score	0.6889	0.7234	0.8196
MCC	0.6474	0.6831	0.7878
Cohen's Kappa	0.6435	0.6809	0.7875

All additional curves for LSTM and CNN are available in the appendix.

4.2 Limitations

These models have been run in Google-Colab environment, involving a limitation of RAM ($12,7$Gb) and space. This limitation force us to constrain the amount of data used for training, explaining why undersampling was done, as well as removing the extra rare cases of cases instead of trying to oversample them. The attack classification can appear much less effective than other models reviewed in this paper, this can be due to the dataset's size and the environment greatly limiting data augmentation. In addition, the dataset comes from a controlled APT simulation environment with limited scenarios, which may not capture the full variability of real-world attacker behaviour, tooling, or network noise.

4.3 Conclusion

This study evaluated the performance of a two-stage deep learning pipeline for cyberattack detection and MITRE ATT&CK tactic classification from logs. Motivated by the limitations of traditional signature-based IDS and the growing sophistication of APT campaigns, we framed the problem as a time-series classification task over sliding windows of security events, allowing the models to capture the temporal patterns that characterise multi-step attack behaviour rather than relying on individual event-level signals.

Our pipeline with two complementary models gives really encouraging results. The first, a two-layer LSTM, performs binary attack detection on the full event stream and achieves approximately $97,75\%$ accuracy and an F1-score of 0.97 for both classes, demonstrating that sequential modelling of SIEM logs can reliably detect malicious from benign activity. The second, a 1D-CNN with an attention mechanism, classifies the detected attack sequences into one of the MITRE ATT&CK tactics with an accuracy of $81,30\%$, giving security analysts an direct and structured understanding of what an adversary is attempting to do.

The use of a strict chronological train/validation/test split throughout ensures that reported metrics reflect true generalisation to future, unseen data, making the evaluation credible and directly comparable to real deployment conditions.

4.4 Appendix

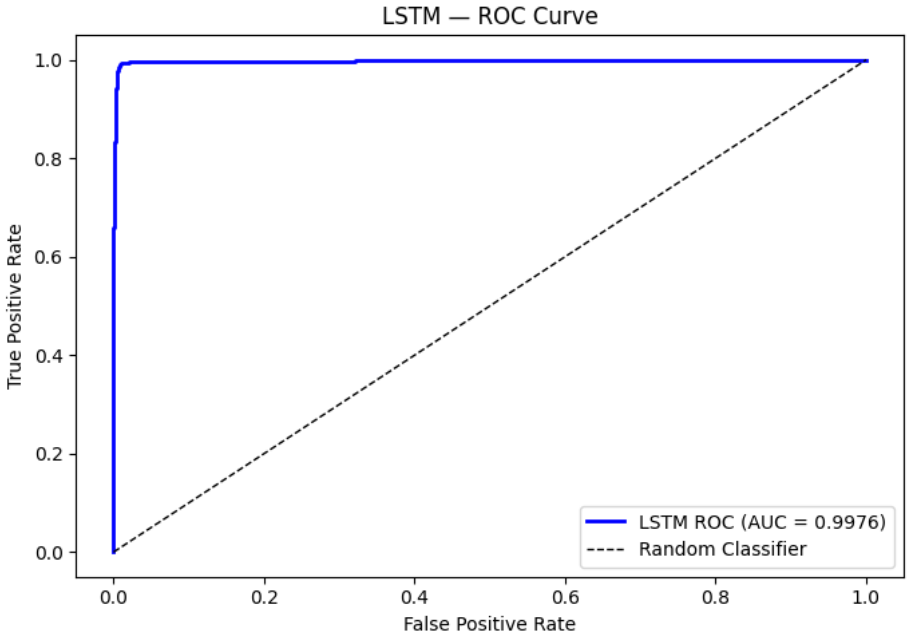

Figure 5 : LSTM ROC curve

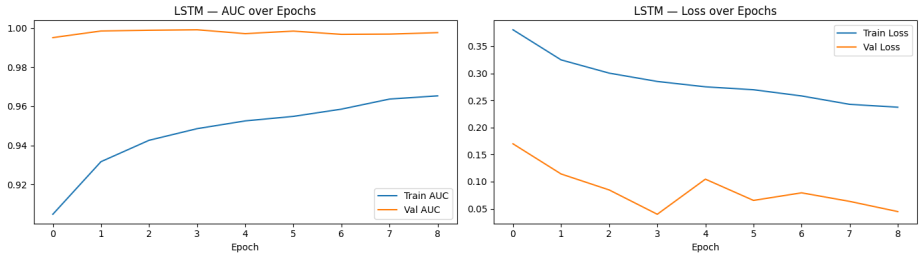

Figure 6 : LSTM curves

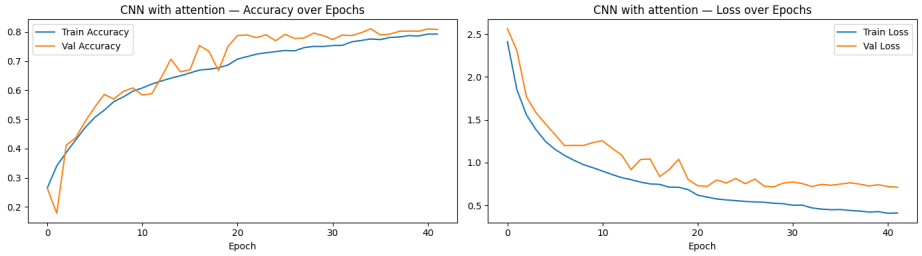

Figure 7 : CNN curves

References

1. L. Laboratory, "1998 darpa intrusion detection evaluation dataset," *Massachusetts Institute of Technology*, 1998.
2. N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.
3. I. S. et al., "4th international conference on information systems security and privacy (icissp)," in *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*, 2018.
4. S. S. Karim, M. Afzal, W. Iqbal, and D. A. Abri, "Advanced persistent threat (apt) and intrusion detection evaluation dataset for linux systems 2024," *Data in Brief*, vol. 54, p. 110290, 2024.
5. M. Mozaffari, A. Yazdinejad, and A. Dehghantanha, "Windows-apt 2025: A dataset for apt-inspired attack scenarios on windows systems," *Data in Brief*, vol. 65, p. 112569, 2026.
6. M. M. Hassan, A. Gumaei, A. Alsanad, M. Alrubaian, and G. Fortino, "A hybrid deep learning model for efficient intrusion detection in big data environment," *Information Sciences*, vol. 513, pp. 386–396, 2020.
7. A. M. Alashjaee, "Deep learning for network security: an attention-cnn-lstm model for accurate intrusion detection.," *Scientific reports*, vol. 15, 2025.
8. A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "Cnn-lstm: Hybrid deep neural network for network intrusion detection system," *IEEE Access*, vol. 10, pp. 99837–99849, 2022.
9. S. Meganathan, A. Sumathi, S. S. M. Shah, and R. Rajakumar, "Real-time intrusion detection using adaptive sliding windows for concept drift," *SSRG International Journal of Electronics and Communication Engineering*, vol. 12, no. 8, pp. 123–137, 2025.
10. W. Dai, X. Li, W. Ji, and S. He, "Network intrusion detection method based on cnn-bilstm-attention model," *IEEE Access*, vol. 12, pp. 53099–53111, 2024.